Залікова робота з предмету «Прикладні задачі аналізу великих даних»

Захаров Дмитро, Варіант 5

31 травня, 2025

Зміст

1 Задача 1 **2**
- 1.1 Загальне означення . . . 2
- 1.2 Методи середнього рівня . . . 2
  - 1.2.1 Середнє значення . . . 2
  - 1.2.2 Медіана та мода . . . 3
- 1.3 Методи розсіювання (варіабельність) . . . 4
  - 1.3.1 Дисперсія, стандартне відхилення . . . 4
  - 1.3.2 Стандартна помилка . . . 5
  - 1.3.3 Коефіцієнт ексцесу (Kurtosis) та асиметрії (Skewness) . . . 5
  - 1.3.4 Квантилі та довірчі інтервали . . . 7

2 Задача 2 **8**
- 2.1 Загальний опис набору даних . . . 8
- 2.2 Згладжування даних . . . 8
- 2.3 Дескриптивна статистика . . . 10
  - 2.3.1 Функції густини . . . 10
  - 2.3.2 Статистичні показники . . . 13
  - 2.3.3 Показники по роках . . . 19
- 2.4 Спектральний аналіз . . . 20
- 2.5 Фазовий аналіз динамічної системи . . . 26
- 2.6 Кореляційний аналіз . . . 28

1 Задача 1

Умова 1.1. Що включає описова статистика? Як ці показники пов'язані між собою і про що говорять? *(15 балів).*

1.1 Загальне означення

Definition 1.2. Описова статистика це сукупність методів, які використовуються для систематизації та представлення даних у вигляді графіків і таблиць, а також числовим узагальненням за допомогою ключових статистичних показників.

Помітимо, що вона не робить жодних висновків про генеральну сукупність, а лише описує наявні дані. Зазвичай, використовуються наступні три методи агрерування та представлення даних:

- **Графічні методи** — це діаграми, гістограми, графіки та інші візуальні представлення даних.
- **Табличні методи** — рядки і стовпці, в якій в певному порядку викладається певна статистична інформація про задані події чи процеси.
- **Статистичні показники** — це числові характеристики, які узагальнюють дані, наприклад, середнє значення, медіана, мода, дисперсія (будемо розглядати більш детально надалі).

1.2 Методи середнього рівня

Які саме статистичні показники використовуються? Розглянемо основні методи середнього рівня, що усереднюють певні показники даних.

1.2.1 Середнє значення

Якщо вважати вибірку даних $X_1, \ldots, X_n \sim X$ елементами, що розподілені незалежно однаково за певним розподілом X (на практиці, неперервним з густиною ρ_X), то нас може цікавити, наприклад, середнє значення цих даних. Для цього, в теорії ймовірностей, ми використовуємо **математичне сподівання**, яке в загальному випадку для величини X з кумулятивною функцією розподілу F_X над ймовірнісним простором Ω визначається як $\mathbb{E}[X] \triangleq \int_\Omega x dF_X(x)$. Незміщенною та асимптотично стійкою оцінкою математичного сподівання розподілу є *вибіркове середнє* $\overline{X} \triangleq \frac{1}{n}\sum_{i=1}^n X_i$, тобто це означає, що $\mathbb{E}[\overline{X}] = \mu$ та $\overline{X} \to \mu$[1], де μ — математичне сподівання розподілу X. Проте, у випадках, коли структура даних або природа задачі вимагає врахування певних особливостей, ми можемо використовувати і інші види

[1]Якщо ще більш формально, то $\forall \varepsilon > 0 : \lim_{n\to\infty} \Pr\left[\left|\overline{X}_n - \mathbb{E}[X]\right| > \varepsilon\right] = 0$, де $\overline{X}_n \triangleq \frac{1}{n}\sum_{i=1}^n X_i$.

середнього значення:

- Гармонійне середнє визначається як $\overline{X}_H \triangleq \frac{n}{\sum_{i=1}^n X_i^{-1}}$ і використовується, коли дані представляють собою швидкості або інші величини, що мають зворотну пропорційність. Наприклад, якщо ви їдете n відстаней ℓ з різними швидкостями v_i, то середня швидкість $\overline{v} = \frac{\ell}{T}$, де $T = \sum_{i=1}^n \frac{\ell}{v_i}$, буде дорівнювати саме гармонійному середньому $\overline{v}_H$. Також її можна зустріти для обрахунку метрики F_1-score в задачах класифікації, де вона використовується для обчислення середньої гармонійної точності та повноти (precision та recall).
- Геометричне середнє визначається як $\overline{X}_G \triangleq \left(\prod_{i=1}^n X_i\right)^{1/n}$ і застосовується для даних, що мають відношення до відсотків або відносних змін.
- Квадратичне середнє визначається як $\overline{X}_Q \triangleq \sqrt{\frac{1}{n}\sum_{i=1}^n X_i^2}$ і використовується для даних, що мають відношення до енергії та потенціалу (наприклад, в електриці або механіці). Також, саме це середнє часто використовується в машинному навчанні для оцінки якості моделей, наприклад, в середньоквадратичній помилці (MSE).
- Степеневе середнє визначається як $\overline{X}_p \triangleq \left(\frac{1}{n}\sum_{i=1}^n X_i^p\right)^{1/p}$, де p — це параметр, що визначає ступінь середнього.
- Взважене середнє: якщо кожному елементу вибірки X_i відповідає вага w_i (причому, без обмеження загальності, нехай $\sum_{i=1}^n w_i = 1$), то взагальне середнє визначається як $\overline{X}_W \triangleq \sum_{i=1}^n w_i X_i$. Це середнє використовується, коли дані мають різну важливість або, наприклад, мають різну точність. Зокрема, якщо ми знаємо дисперсію кожного показника X_i, то можна задати $w_i \propto \frac{1}{\mathrm{Var}[X_i]+\varepsilon^2}$ для малого ε.

1.2.2 Медіана та мода

Окрім середніх значень, для опису середнього рівня часто використовуються **медіана** та **мода**. Медіана визначається як значення, яке розділяє вибірку даних на дві рівні частини, тобто половина значень менша або дорівнює медіані, а інша половина більша або дорівнює їй. Показує центральне розташування даних і менш чутлива до викидів і асиметрії, ніж середнє. Більш формально, для величини X, медіаною називаємо таке значення M, для якого $F_X(M) = \Pr[X \leq M] = 0.5$. В свою чергу модою M_0 називаємо таке значення, яке має найбільшу ймовірність в розподілі X, тобто $M_0 = \arg\max_{\omega\in\Omega} \rho_X(\omega)$ для густини розподілу ρ_X. Згідно означенню, показує, яке значення найчастіше зустрічається в даних. Особливо корисна для категоріальних даних або даних з дискретним розподілом.

1.3 Методи розсіювання (варіабельність)

1.3.1 Дисперсія, стандартне відхилення

До методів розсіювання відносяться статистичні показники, які вимірюють, наскільки дані розкидані навколо середнього значення. Це важливо для розуміння того, наскільки дані варіюються — наприклад, для визначення точності вимірювань або для оцінки ризиків в фінансових операціях.

Перша, найпростіша та найпоширеніша міра розсіювання — це **дисперсія**, яка визначена як $\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2]$ — тобто буквально середнє квадратичне відхилення від середнього значення. У вигляді інтегралу для неперервного розподілу вона виглядає як $\mathrm{Var}[X] = \int_\Omega (x - \mathbb{E}[X])^2 dF_X$. Дуже часто використовується **стандартне відхилення**, яке визначається як $\sigma_X = \sqrt{\mathrm{Var}[X]}$, що дозволяє зберегти одиниці виміру та зробити іноді інтерпретацію більш зрозумілою: наприклад, якщо ми маємо дані про висоту людей, то дисперсія буде мати розмірність m^2, в той час як стандартне відхилення буде просто в метрах, що більш інтуїтивно. Геометрично ми проілюструємо ці поняття на прикладі нормального розподілу: див. Рисунок 1.

Нормальний розподіл з різними дисперсіями σ^2

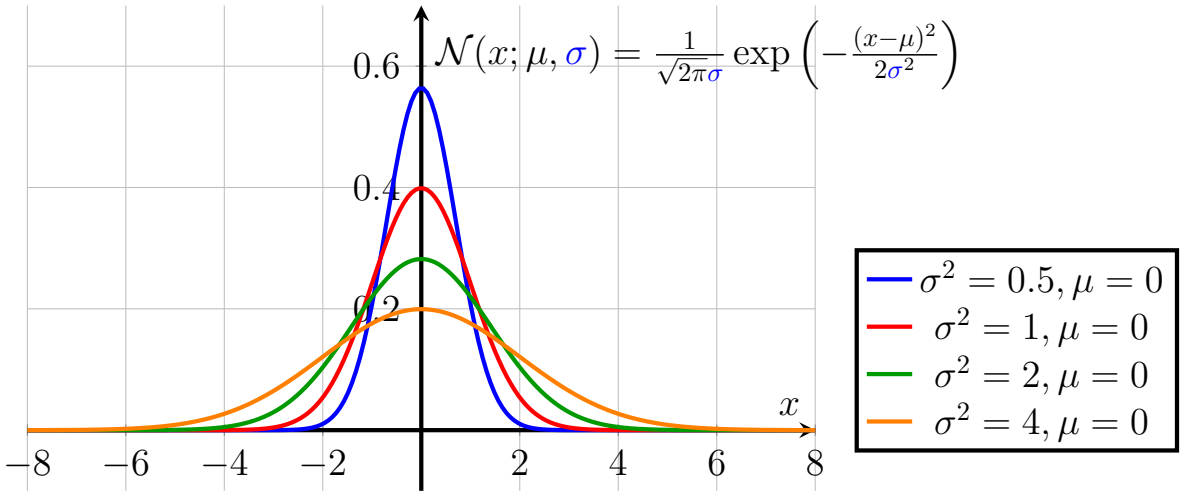


Рис. 1: Нормальні розподіли $\mathcal{N}(x;\mu,\sigma^2)$ з різними дисперсіями σ^2 та нульовим математичним сподіванням $\mu = 0$. Як бачимо, чим більша дисперсія, тим ширше розподілені дані навколо середнього значення.

Щоб обрахувати дисперсію вибірки даних $X_1, \ldots, X_n$, ми можемо використовувати *вибіркову дисперсію*, яка визначається як

$$\widehat{\sigma}^2 \triangleq \frac{1}{n-1} \sum_{i=1}^{n} (X_i - \overline{X})^2,$$

Ця оцінка є незміщеною, тобто $\mathbb{E}[\widehat{\sigma}^2] = \text{Var}[X]$, і асимптотично стійкою, тобто $\widehat{\sigma}^2 \xrightarrow{\rho} \text{Var}[X]$ при $n \to \infty$. Вона використовується для оцінки дисперсії генеральної сукупності на основі вибірки даних.

1.3.2 Стандартна помилка

Термін був вперше введений в 1897 році Удні Юлом і визначений для розподілу з дисперсією σ^2 як $\text{SD}(\mu) \triangleq \frac{\sigma}{\sqrt{n}}$. Проте, оскільки ми ніколи не знаємо дисперсію генеральної сукупності, то ми використовуємо $\text{SE}(\overline{X}) \triangleq \frac{\widehat{\sigma}}{\sqrt{n}}$. Цей показник береться з центральної граничної теореми, яка стверджує, що при достатньо великому розмірі вибірки n розподіл вибіркового середнього $\overline{X}$ буде наближатися до нормального розподілу $\mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$, де як раз таки $\frac{\sigma^2}{n}$ є дисперсією вибіркового середнього. Далі цей параметр може бути використаний для побудови довірчих інтервалів, перевірки гіпотез (наприклад, z-тести/t-тести) тощо.

1.3.3 Коефіцієнт ексцесу (Kurtosis) та асиметрії (Skewness)

Коефіцієнт ексцесу $\kappa[X]$ показує "крутість" підвищення кривої розподілу X відносно нормального розподілу. Він визначається як момент четвертого порядку стандартної величини $Z = \frac{X - \mathbb{E}[X]}{\sqrt{\text{Var}[X]}}$:

$$\kappa[X] \triangleq \mathbb{E}[Z^4] = \frac{\mathbb{E}[(X - \mathbb{E}[X])^4]}{\text{Var}[X]^2} = \frac{\mu_4}{\sigma^4},$$

де $\mu_4 = \mathbb{E}[(X - \mathbb{E}[X])^4]$ — це четвертий центральний момент величини X. Оскільки для стандартного нормального розподілу $Z \sim \mathcal{N}(0, 1)$, маємо $\kappa[Z] = 3$, то часто у якості коефіцієнта ексцесу беруть значення $\kappa[X] - 3$, яке буде дорівнювати нулю для нормального розподілу і більше/менше нуля для розподілів з більшою/меншою крутістю відносно нормального розподілу: це проілюстровано на Рисунку 2.

Порівняння розподілів з різним коефіцієнтом ексцесу $\kappa[X]$

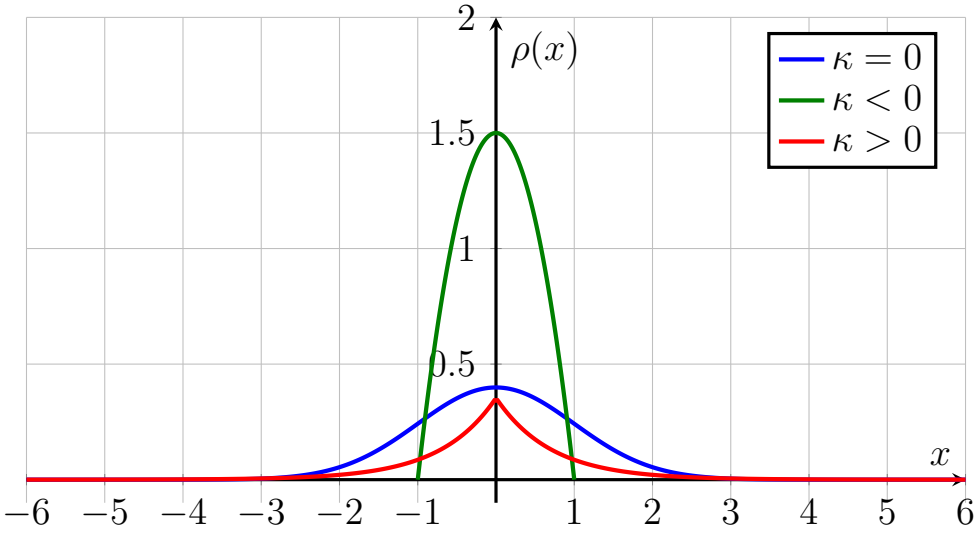


Рис. 2: Порівняння розподілів з різним коефіцієнтом ексцесу. Як бачимо, розподіл з більшим ексцесом має більш гострий пік та довші хвости, в той час як розподіл з меншим ексцесом має більш пласку криву.

Коефіцієнт асиметрії $\gamma[X]$ показує, наскільки розподіл X є асиметричним відносно середнього значення. Він визначається як

$$\gamma[X] \triangleq \frac{\mathbb{E}[(X - \mathbb{E}[X])^3]}{\mathrm{Var}[X]^{3/2}} = \frac{\mu_3}{\sigma^3},$$

де $\mu_3 = \mathbb{E}[(X - \mathbb{E}[X])^3]$ — це третій центральний момент величини X. Якщо $\gamma[X] > 0$, то розподіл має позитивну асиметрію (правий хвіст довший), якщо $\gamma[X] < 0$, то розподіл має негативну асиметрію (лівий хвіст довший), а якщо $\gamma[X] = 0$, то розподіл симетричний. Це проілюстровано на Рисунку 3.

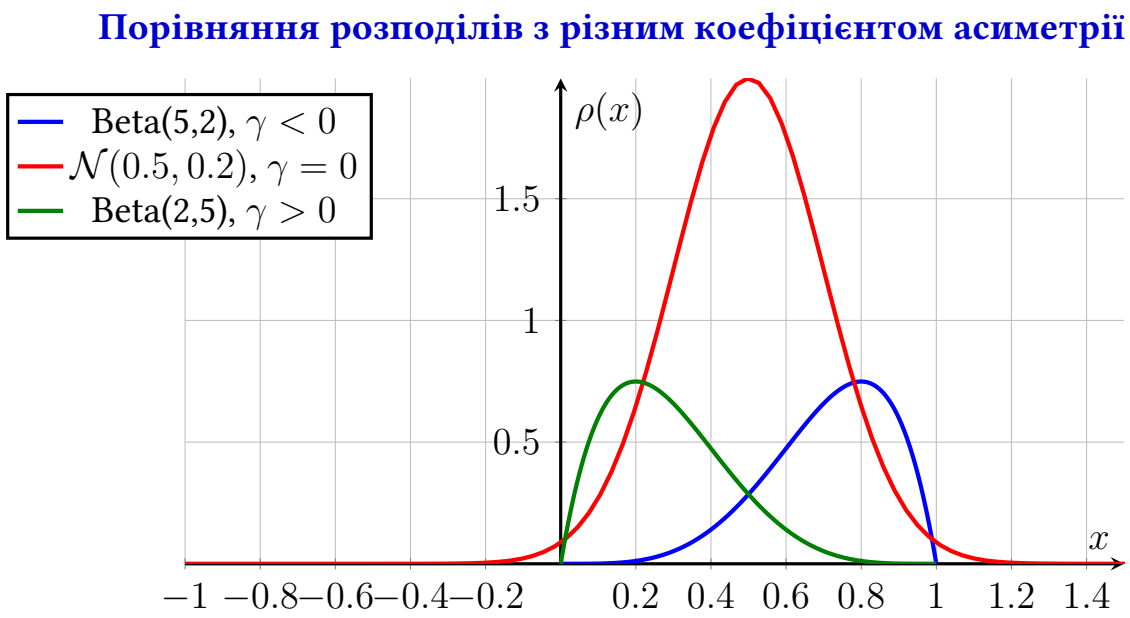


Рис. 3: Ілюстрація від'ємного, нульового і додатнього skewness. Як бачимо, розподіл з від'ємним асиметричним має довший хвіст зліва, розподіл з додатнім асиметричним має довший хвіст справа, а розподіл з нульовим асиметричним симетричний.

1.3.4 Квантилі та довірчі інтервали

Формально квантиллю $\alpha \in (0, 1)$ називають таке значення q_α, що задовільняє $q_\alpha = \inf\{x \in \mathbb{R} : F_X(x) \geq \alpha\}$. Фактично, це значення, яке розділяє розподіл на дві частини: перша частина містить α частину даних, а друга частина містить $1 - \alpha$ частину даних. Зокрема, медіана є $q_{0.5}$. Значення $q_{0.25}$ називають **першим квартилем**, $q_{0.75}$ — **третім квартилем**.

Довірчим інтервалом, у свою чергу, для параметра θ з довірчою ймовірністю $1 - \alpha$ називають такий інтервал $\mathcal{I}_{\theta,\alpha}$, що

$$\Pr[\theta \in \mathcal{I}_{\theta,\alpha}] = 1 - \alpha.$$

Наприклад, якщо нам треба оцінити математичне сподівання μ для нормального розподілу $X \sim \mathcal{N}(\mu, \sigma^2)$ з відомою дисперсією σ (скажімо, X описує вимірювання з відомою похибкою) з кумулятивною функцією розподілу F_X, маючи вибірку $X_1, \ldots, X_n \sim \mathcal{N}(\mu, \sigma^2)$, то ми можемо побудувати довірчий інтервал з довірчою ймовірністю $1 - \alpha$ як:

$$\mathcal{I}_{\mu,\alpha} = (\overline{X} - z_{1-\alpha/2} \cdot \mathrm{SE}(\overline{X}), \overline{X} + z_{1-\alpha/2} \cdot \mathrm{SE}(\overline{X})),$$

де $z_{1-\alpha/2}$ — це квантиль стандартного нормального розподілу (тобто $\Pr[Z \leq z_{1-\alpha/2}] = 1 - \frac{\alpha}{2}$).

2 Задача 2

Умова 2.1. Статистична обробка екологічної інформації. *(25 балів).*

2.1 Загальний опис набору даних

Було запропоновано провести аналіз даних про забруднення повітря в різних містах України. В моєму випадку, я аналізував дані з Хмельницького за наступним посиланням:

https://mepr.gov.ua/diyalnist/napryamky/ekologichnyj-monitoryng/stan-atmosfernogo-povitrya/hmelnytskyj/

Дані містять інформацію про рівень концентрації забруднюючих речовин в атмосферному повітрі, таких як діоксид азоту (NO_2), фенол тощо, в різні моменти часу. Приклад даних виглядає наступним чином:

Число	Завислі речовини	Діоксид сірки	Оксид вуглецю	Діоксид азоту	Оксид азоту	Фенол	Хлористий водень	Аміак	Формальдегід
2/8/2020	0.3	0.6	1.5	1.4	0.4	1.2	0.1	0.1	1.5
2/9/2020									
2/10/2020	0.3	0.4	1.6	1.2	0.3	0.9	0.1	0.1	1.1
2/11/2020	0.2	0.5	1.5	1.2	0.3	1	0.3	0.4	1.6
2/12/2020	0.2	0.6	1.5	1	0.3	1	0.1	0.4	1
2/13/2020	0.4	0.4	1.5	1.3	0.4	0.9	0.1	0.4	1
2/14/2020	0.5	0.5	1.4	1.1	0.3	0.5	0.1	0.3	1.2
2/15/2020	0.4	0.5	1.4	1	0.3	0.6	0.2	0.2	1.2
2/16/2020									
2/17/2020	0.3	0.4	1.4	1	0.2	0.8	0.1	0.3	1.8
2/18/2020	0.3	0.3	1.3	1.7	0.5	1.7	0.1	0.3	1.5
2/19/2020	0.4	0.4	1.3	1	0.3	1	0.1	0.2	1
2/20/2020	0.4	0.4	1.1	1.1	0.2	0.8	0.3	0.2	1.1
2/21/2020	0.4	0.5	1.1	0.7	0.1	1	0.2	0.3	1
2/22/2020	0.4	0.5	1.1	0.8	0.1	0.9	0.2	0.2	1.2
2/23/2020									
2/24/2020	0.3	0.4	1.2	1.3	0.3	1.0	0.0	0.2	1.4
2/25/2020	0.3	0.5	1.2	1.0	0.2	0.9	0.1	0.2	0.8

Рис. 4: Приклад даних про забруднення повітря в Хмельницькому.

Міст сирої таблички даних прикріплений до роботи під назвою khmelnytskyi.csv.

2.2 Згладжування даних

Для аналізу даних про забруднення повітря було застосовано метод згладжування даних, який дозволяє заповнити пропуски в даних та зменшити вплив шуму. Окрім того, деякі методи аналізу даних вимагають наявності гладких даних, тому згладжування є важливим етапом підготовки даних.

Для цього ми застосували наступний код на Python. Спочатку ми просто завантажуємо дані з CSV-файлу, конвертуємо колонки в числові значення та парсимо дати. Потім, виділяємо колонки з забруднюючими речовинами.

```
def load_and_preprocess_data(file_path: Path):
    """
```

```
    Loads the CSV data, converts columns to numeric, and parses dates.
    """

    try:
        df = pd.read_csv(file_path)
    except FileNotFoundError:
        print(f"Error: The file '{file_path}' was not found.")
        return None, None

    # Convert 'Date' column to datetime objects
    df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
    # Remove rows where date conversion failed
    df = df.dropna(subset=['Date'])

    if df.empty:
        print("Error: No valid dates found after parsing. DataFrame is empty.")
        return None, None

    df = df.set_index('Date')

    # Identify pollutant columns
    pollutant_columns = df.columns.tolist()

    # Convert pollutant columns to numeric
    for col in pollutant_columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

    return df, pollutant_columns
```

Також, додаємо метод для згладжування даних:

```
def fill_all_gaps(df, pollutant_columns, method='linear'):
    """
    Fills all gaps in the specified columns using interpolation.
    """

    df_filled_all = df.copy()
    for col in pollutant_columns:
        df_filled_all[col] = df_filled_all[col].interpolate(method=method)

    return df_filled_all
```

Тут використовується метод лінійної інтерполяції для заповнення пропусків в даних (себто, якщо пропущене значення для часу $\tau \in (t_0, t_1)$ де відомі послідовні значення $x(t_0)$ та $x(t_1)$, то ми заповнюємо його наступнм чином: $x(\tau) = x(t_0) + \frac{(\tau - t_0)}{t_1 - t_0}(x(t_1) - x(t_0))$). Помітимо, що можна виокристовувати і більш складні методи згладжування, наприклад, *сплайн-інтерполяцію* або

поліноміальну регресію, але в нашому випадку лінійна інтерполяція достатня для згладжування даних.

Результат згладжування даних збережено у `khmelnytskyi_filled_all_gaps.csv`. Приклад згладжених даних зображено на Рисунку 5.

```
2023-12-04,0.52,0.3293,0.555,0.8225,0.5296,0.3167,0.2111,0.1631,0.3667
2023-12-05,0.4233,0.2775,0.525,0.6634,0.369,0.35,0.1912,0.1238,0.3222
2023-12-06,0.4267,0.275,0.5575,0.5116,0.3715,0.3056,0.1661,0.1306,0.3667
2023-12-07,0.3667,0.2318,0.57,0.6119,0.3296,0.4444,0.1525,0.1306,0.4944
2023-12-08,0.3667,0.2392,0.61,0.7212,0.3365,0.4889,0.1262,0.1438,0.5722
2023-12-09,0.3667,0.2352,0.6008,0.5103,0.3027,0.3222,0.131,0.0975,0.4056
2023-12-10,0.3967000000000005,0.24435,0.5804,0.6431,0.36915,0.31945,0.12705,0.180649999
2023-12-11,0.4267,0.2535,0.56,0.7759,0.4356,0.3167,0.1231,0.2638,0.3667
2023-12-12,0.2667,0.193,0.4533,0.7253,0.3894,0.2944,0.2552,0.1519,0.4111
2023-12-13,0.3783,0.212,0.5258,0.69,0.4148,0.3111,0.3984,0.1306,0.45
2023-12-14,0.3793923076923077,0.21201923076923077,0.5244884615384616,0.6843499999999999,
2023-12-15,0.3804846153846154,0.21203846153846154,0.5231769230769231,0.6787,0.4064846153
2023-12-16,0.3815769230769231,0.2120576923076923,0.5218653846153847,0.6730499999999999,0
2023-12-17,0.3826692307692308,0.21207692307692308,0.5205538461538461,0.6674,0.3981692307
2023-12-18,0.3837615384615385,0.21209615384615385,0.5192423076923077,0.66175,0.394011538
2023-12-19,0.38485384615384616,0.21211538461538462,0.5179307692307693,0.6561,0.389853846
2023-12-20,0.3859461538461539,0.21213461538461537,0.5166192307692308,0.65045,0.385696153
2023-12-21,0.38703846153846155,0.21215384615384614,0.5153076923076924,0.6447999999999999
2023-12-22,0.3881307692307692,0.2121730769230769,0.5139961538461539,0.63915,0.3773807692
2023-12-23,0.38922307692307695,0.21219230769230768,0.5126846153846154,0.6335,0.373223076
2023-12-24,0.3903153846153846,0.21221153846153845,0.511373076923077,0.62785,0.3690653846
2023-12-25,0.39140769230769235,0.21223076923076922,0.5100615384615385,0.6222,0.364907692
2023-12-26,0.3925,0.21225,0.50875,0.6165499999999999,0.36075,0.39445,0.28114999999999996
2023-12-27,0.3935923076923077,0.21226923076923077,0.5074384615384616,0.6109,0.3565923076
```

Рис. 5: Приклад згладжених даних.

2.3 Дескриптивна статистика

2.3.1 Функції густини

Тут ми можемо обрахувати деякі статистичні показники для кожної забруднюючої речовини, наприклад, середнє значення, медіану, моду, дисперсію, стандартне відхилення, коефіцієнт асиметрії та ексцесу. Також, ми робимо цей обрахунок по роках, щоб побачити, як змінюється рівень забруднення протягом часу. Але для початку, давайте просто відобразимо функції густини для кожної забруднюючої речовини, щоб побачити їх розподіл. Для цього виконаємо наступний код на Python:

```python
def plot_density_functions(df, pollutant_columns, output_dir="plots/density_functions"):
    """
    Generates and saves probability density function (PDF/KDE) plots
    for each pollutant in separate files.
    """

    if df.empty:
        print("DataFrame is empty. No density plots will be generated.")
        return
```

```
    # Ensure the output directory exists
    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)
    print(f"Ensured directory exists: {output_path}")

    # Iterate through each pollutant column
    for col in pollutant_columns:
        plt.figure(figsize=(10, 6))
        data_series = df[col].dropna()

        # Check if there is data left to plot
        if data_series.empty:
            print(f"No data for pollutant '{col}'. Skipping density plot.")
            plt.close()
            continue

        # Create the Kernel Density Estimate plot using seaborn
        sns.kdeplot(data_series, fill=True, bw_adjust=0.75)

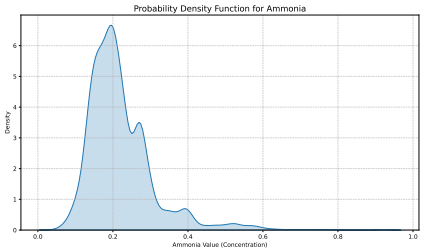
        # Customize the plot
        plt.title(f'Probability Density Function for {col}', fontsize=14)
        plt.xlabel(f'{col} Value (Concentration)', fontsize=10)
        plt.ylabel('Density', fontsize=10)
        plt.grid(True, linestyle='--', alpha=0.6)
        plt.tight_layout()

        # Prepare filename
        safe_col_name = col.replace(' ', '_').replace('/', '_').lower()
        plot_filename = f"density_{safe_col_name}.pdf"
        full_plot_path = output_path / plot_filename

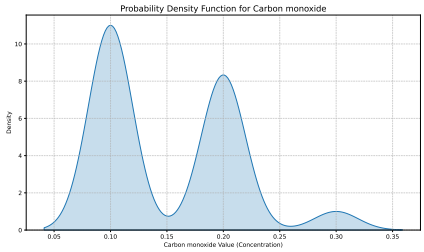
        # Save the plot
        try:
            plt.savefig(full_plot_path, dpi=300, format='pdf', bbox_inches='tight')
            print(f"Saved density plot to: {full_plot_path}")
        except Exception as e:
            print(f"Error saving plot for {col}: {e}")

        plt.close()
```

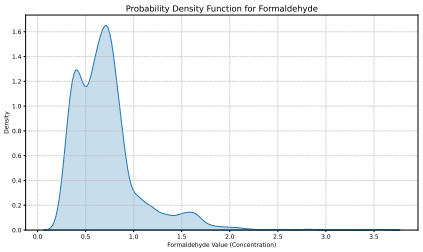
Отримаємо наступні графіки функцій густини для кожної забруднюючої речовини: див. Рисунок 6. Як бачимо, деякі забруднюючі речовини мають розподіл, близький до нормального, в той час як інші мають більш складний розподіл з декількома піками. Це може свідчити про наявність різних джерел забруднення або про сезонні коливання в рівні забруднення повітря (визначення цих джерел та сезонних коливань виходить за межі цієї роботи і взагалі роботи математика/статистика).



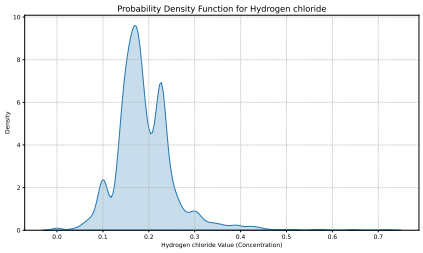
Ammonia



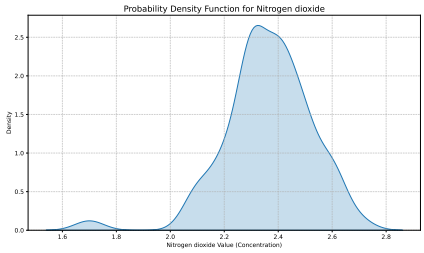
Carbon Monoxide



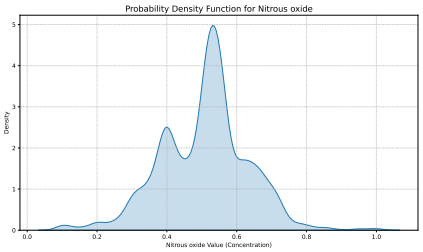
Formaldehyde



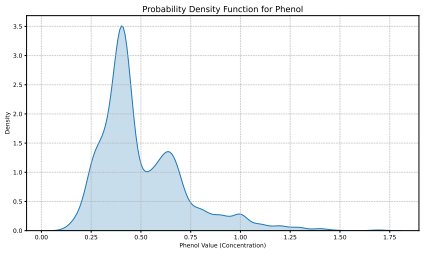
Hydrogen Chloride



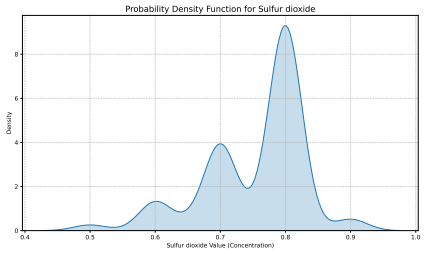
Nitrogen Dioxide



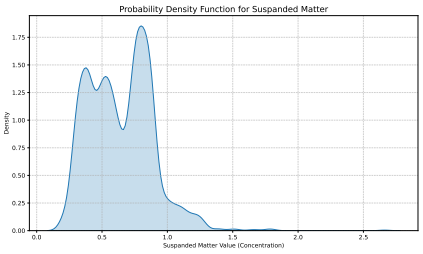
Nitrous Oxide



Phenol



Sulfur Dioxide



Suspended Matter

Рис. 6: Графіки функцій густини для кожної забруднюючої речовини

2.3.2 Статистичні показники

Тепер ми можемо обрахувати деякі статистичні показники для кожної забруднюючої речовини. Для цього ми можемо використати наступний код на Python:

```
def calculate_descriptive_stats(df, pollutant_columns):
    """
    Calculates descriptive statistics for each pollutant, grouped by year and overall.
    """

    all_stats = []

    if df.empty or df.index.empty:
        print("Warning: DataFrame is empty or has no valid DatetimeIndex. Cannot calcula
        return pd.DataFrame()

    if not isinstance(df.index, pd.DatetimeIndex):
        print("Warning: Index is not a DatetimeIndex. Cannot extract year.")

        try:
            df.index = pd.to_datetime(df.index)
        except Exception as e:
            print(f"Error converting index to DatetimeIndex: {e}")
            return pd.DataFrame()

    df_with_year = df.copy()
    df_with_year['Year'] = df_with_year.index.year

    if df_with_year['Year'].isnull().all():
        print("Warning: Could not extract valid years from the DataFrame index.")
        years = []
    else:
        years = sorted(df_with_year['Year'].dropna().unique().astype(int))

    for col in pollutant_columns:
        # Overall statistics
        data_series = df_with_year[col].dropna()
        if not data_series.empty:
            mode_result = stats.mode(data_series, keepdims=True)
            mode_val = mode_result.mode[0] if mode_result.mode.size > 0 else np.nan

            stats_dict = {
                'Pollutant': col,
                'Year': 'Overall',
                'Mean': data_series.mean(),
                'Median': data_series.median(),
                'Mode': mode_val,
                'Min': data_series.min(),
```

```
                'Max': data_series.max(),
                'Kurtosis': data_series.kurtosis(),
                'Skewness': data_series.skew(),
                'Std Dev': data_series.std()
            }
            all_stats.append(stats_dict)

        # Per-year statistics
        for year in years:
            year_data = df_with_year[df_with_year['Year'] == year][col].dropna()
            if not year_data.empty:
                mode_result_year = stats.mode(year_data, keepdims=True)
                mode_val_year = mode_result_year.mode[0] if mode_result_year.mode.size >

                stats_dict_year = {
                    'Pollutant': col,
                    'Year': str(year),
                    'Mean': year_data.mean(),
                    'Median': year_data.median(),
                    'Mode': mode_val_year,
                    'Min': year_data.min(),
                    'Max': year_data.max(),
                    'Kurtosis': year_data.kurtosis(),
                    'Skewness': year_data.skew(),
                    'Std Dev': year_data.std()
                }
                all_stats.append(stats_dict_year)

    stats_df = pd.DataFrame(all_stats)
    if not stats_df.empty:
        stats_df['Year'] = stats_df['Year'].astype(str)
        stats_df = stats_df.set_index(['Pollutant', 'Year'])

    return stats_df
```

Як бачимо, ми обраховуємо наступні показники:

- **Середнє значення** (Mean) — середнє арифметичне значення. За деталями див. Розділ 1.2.1.
- **Медіана та модна** (Median & Mode) — за деталями див. Розділ 1.2.2.
- **Мінімальне та максимальне значення** (Min & Max) — найменше та найбільше значення в даних відповідно.
- **Коефіцієнт асиметрії та ексцесу** (Skewness & Kurtosis) — за деталями див. Розділ 1.3.3.
- **Стандартне відхилення** (Std Dev) — міра розсіювання даних навколо середнього значення. За деталями див. Розділ 1.3.1.

Ми зберегли усі показники в CSV-файл `khmelnytskyi_descriptive_stats.csv`, який містить статистику по кожній забруднюючій речовині та по роках. Приклад отриманої таблиці зображено на Рисунку 7.

```
Pollutant,Year,Mean,Median,Mode,Min,Max,Kurtosis,Skewness,Std Dev      You, vor 6 Tagen
Suspanded Matter,Overall,0.6442569458375125,0.6315999999999999,0.3383,0.2,2.6617,2.548824
Suspanded Matter,2020,0.647482493150685,0.5217,0.3,0.2,1.285,-1.0876551330775261,0.496091
Suspanded Matter,2021,0.6921717795042138,0.6677876470588234,0.3883,0.2767,1.6783,0.645930
Suspanded Matter,2022,0.8252330661322644,0.8252330661322644,0.7589617234468937,0.75896172
Suspanded Matter,2023,0.6124513861864931,0.6304375,0.5717,0.2667,1.1167,-0.43410315688634
Suspanded Matter,2024,0.5058537305590584,0.4773777777777778,0.2667,0.2133,1.8,7.738211693
Suspanded Matter,2025,0.5105788690476191,0.41126956521739133,0.3383,0.2683,2.6617,16.4049
Sulfur dioxide,Overall,0.4199430040120361,0.43160671342685364,0.3497,0.1248,0.8,-0.480456
Sulfur dioxide,2020,0.4570883287671233,0.4646070588235294,0.5,0.1925,0.8,0.51140093261801
Sulfur dioxide,2021,0.5207098129958547,0.5268,0.4888,0.2185,0.6617,5.058468582820316,-1.3
Sulfur dioxide,2022,0.48631763527054106,0.48631763527054106,0.43762625250500997,0.4376262
Sulfur dioxide,2023,0.34519980779261367,0.3457,0.2865,0.193,0.4662,-1.1917246931428171,-0
Sulfur dioxide,2024,0.3260253047498949,0.3334115384615385,0.3235,0.1248,0.476,0.425857877
Sulfur dioxide,2025,0.3431005952380952,0.3497,0.3497,0.24,0.4058,2.424492630921804,-1.012
Carbon monoxide,Overall,0.612476479438315,0.5694067134268537,0.525,0.2583,2.3,15.70159067
Carbon monoxide,2020,0.7057428493150685,0.5579317647058823,0.5,0.2583,2.3,2.5952094295551
Carbon monoxide,2021,0.6296566452905811,0.6435994117647058,0.4733,0.3675,1.0583,0.4358431
Carbon monoxide,2022,0.45936973947895793,0.45936973947895793,0.37974929859719436,0.379749
Carbon monoxide,2023,0.6063670040608086,0.58294,0.5358,0.4533,0.8867,1.078553276287321,1.
Carbon monoxide,2024,0.6694288881883145,0.6342,0.5725,0.4108,1.1058,-0.41153309371828906,
Carbon monoxide,2025,0.594359523809524,0.5921000000000001,0.525,0.4958,0.9508,4.229326776
Nitrogen dioxide,Overall,0.9611912738214644,0.9172,0.9172,0.3538,3.0,6.391503689782919,1.
Nitrogen dioxide,2020,1.253581479452055,1.1,1.7,0.4975,3.0,0.35932751499876536,0.91609569
```

Рис. 7: Приклад статистичних показників для забруднюючих речовин в Хмельницькому.

Візуалізуємо ці показники. Найбільш репрезентативним з них є середнє значення по роках, яке показує, як змінюється середній рівень забруднення протягом часу. Для цього ми можемо використати наступний код на Python:

```python
def plot_yearly_means_grouped(stats_df, output_dir="plots"):
    """
    Generates and saves a single grouped bar plot showing mean pollutant levels
    for each year.
    """

    if stats_df.empty:
        print("Statistics DataFrame is empty. No plot will be generated.")
        return

    output_path = Path(output_dir)
    if not output_path.exists():
        os.makedirs(output_path)
        print(f"Created directory: {output_path}")

    try:
        # 1. Prepare data
```

```
        mean_stats = stats_df['Mean'].reset_index()
        mean_stats_yearly = mean_stats[mean_stats['Year'] != 'Overall'].copy()

        if mean_stats_yearly.empty:
            print("No yearly data found to plot.")
            return

        # 2. Ensure 'Year' is numeric for sorting, then convert back to string
        mean_stats_yearly['Year'] = pd.to_numeric(mean_stats_yearly['Year'])
        mean_stats_yearly = mean_stats_yearly.sort_values('Year')
        mean_stats_yearly['Year'] = mean_stats_yearly['Year'].astype(str)

        # 3. Pivot data for grouped bar chart (Years as index, Pollutants as columns)
        plot_data = mean_stats_yearly.pivot(index='Year', columns='Pollutant', values='M

        if plot_data.empty:
            print("Could not pivot data for plotting.")
            return

        # 4. Create the plot
        num_years = len(plot_data.index)
        fig_width = max(14, num_years * 1.5)
        _ = plot_data.plot(kind='bar', figsize=(fig_width, 8), width=0.8)

        # 5. Setting plot design
        plt.title('Mean Pollutant Levels by Year in Khmelnytskyi', fontsize=16)
        plt.xlabel('Year', fontsize=12)
        plt.ylabel('Mean Value (Concentration / Index)', fontsize=12)
        plt.xticks(rotation=45, ha='right')
        plt.legend(title='Pollutants', bbox_to_anchor=(1.02, 1), loc='upper left')
        plt.grid(axis='y', linestyle='--', alpha=0.7)
        plt.tight_layout(rect=[0, 0, 0.88, 1])

        # 6. Define the output filename and path
        plot_filename = "yearly_mean_grouped_by_pollutant.pdf"
        full_plot_path = output_path / plot_filename

        # 7. Save the plot as PDF
        plt.savefig(full_plot_path, dpi=300, format='pdf', bbox_inches='tight')
        plt.close()
        print(f"Saved grouped bar plot to: {full_plot_path}")

    except KeyError as e:
        print(f"Error: Could not find expected column for plotting: {e}")
    except Exception as e:
        print(f"An error occurred while generating the grouped bar plot: {e}")
```

Результат зображено на Рисунку 8.

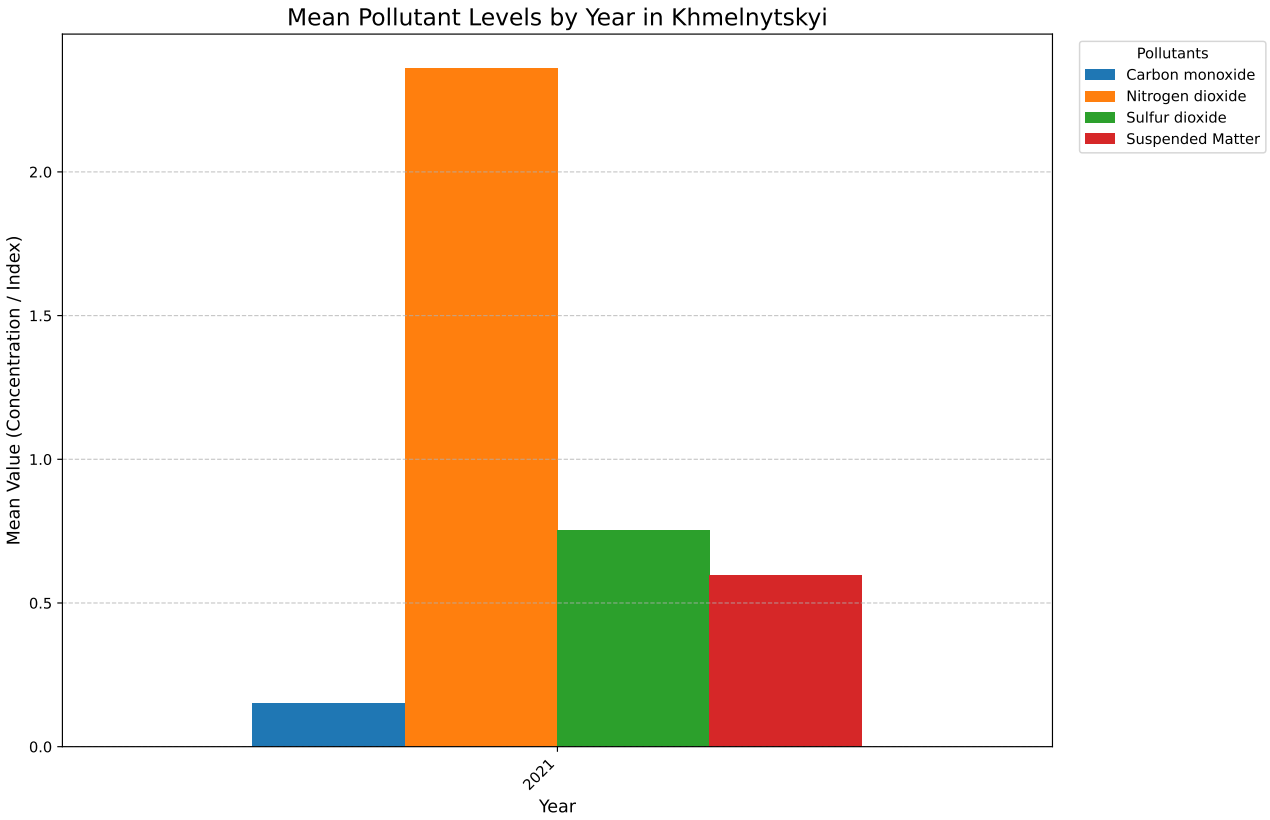


Рис. 8: Груповаий стовпчиковий графік середніх значень забруднюючих речовин по роках. Як бачимо, рівень забруднення повітря в Хмельницькому має тенденцію до зниження протягом 2020—2023, зокрема, рівень формальдегіду та фенолу. Після цього, багато показників коливаються в межах одного рівня (наприклад, той самий фенол або, скажімо, nitrogen dioxide).

Також ми вирішили порівняти середні значення забруднюючих речовин між собою, щоб побачити, які з них є найбільш концентрованими в регіоні Хмельницького. Для цього ми можемо використати наступний код на Python:

```
def plot_overall_means_pdf(stats_df, output_dir="plots"):
    """
    Generates and saves a single plot comparing the 'Overall' mean
    values across all pollutants. Saves as PDF with 300 dpi.
    """

    if stats_df.empty:
        print("Statistics DataFrame is empty. No plot will be generated.")
        return

    output_path = Path(output_dir)
    if not output_path.exists():
```

```
        os.makedirs(output_path)
        print(f"Created directory: {output_path}")

    try:
        if 'Overall' not in stats_df.index.get_level_values('Year'):
            print("No 'Overall' data found in statistics. Skipping plot.")
            return

        # Extract 'Overall' 'Mean' values and sort them
        overall_means = stats_df.xs('Overall', level='Year')['Mean'].sort_values()

        if overall_means.empty:
            print("No 'Overall' 'Mean' data to plot.")
            return

        # Create the plot
        plt.figure(figsize=(12, 8))
        overall_means.plot(kind='barh', colormap='rainbow')
        plt.title('Overall Mean Air Pollutant Values in Khmelnytskyi')
        plt.xlabel('Mean Value (Concentration / Index)')
        plt.ylabel('Pollutant')
        plt.grid(axis='x', linestyle='--', alpha=0.6)
        plt.tight_layout()

        # Define the output filename and path
        plot_filename = "overall_mean_by_pollutant.pdf"
        full_plot_path = output_path / plot_filename

        # Save the plot
        plt.savefig(full_plot_path, dpi=300, format='pdf', bbox_inches='tight')
        plt.close()
        print(f"Saved plot to: {full_plot_path}")
    except KeyError:
        print("Could not find 'Mean' statistic or 'Overall' year for plotting.")
    except Exception as e:
        print(f"An error occurred while plotting: {e}")
```

Результат зображено на Рисунку 9. Як бачимо, найбільш концентрованими забруднюючими речовинами в Хмельницькому є nitrogen dioxide, formaldehyde та suspanded matter, які мають найбільші середні значення серед усіх забруднюючих речовин.

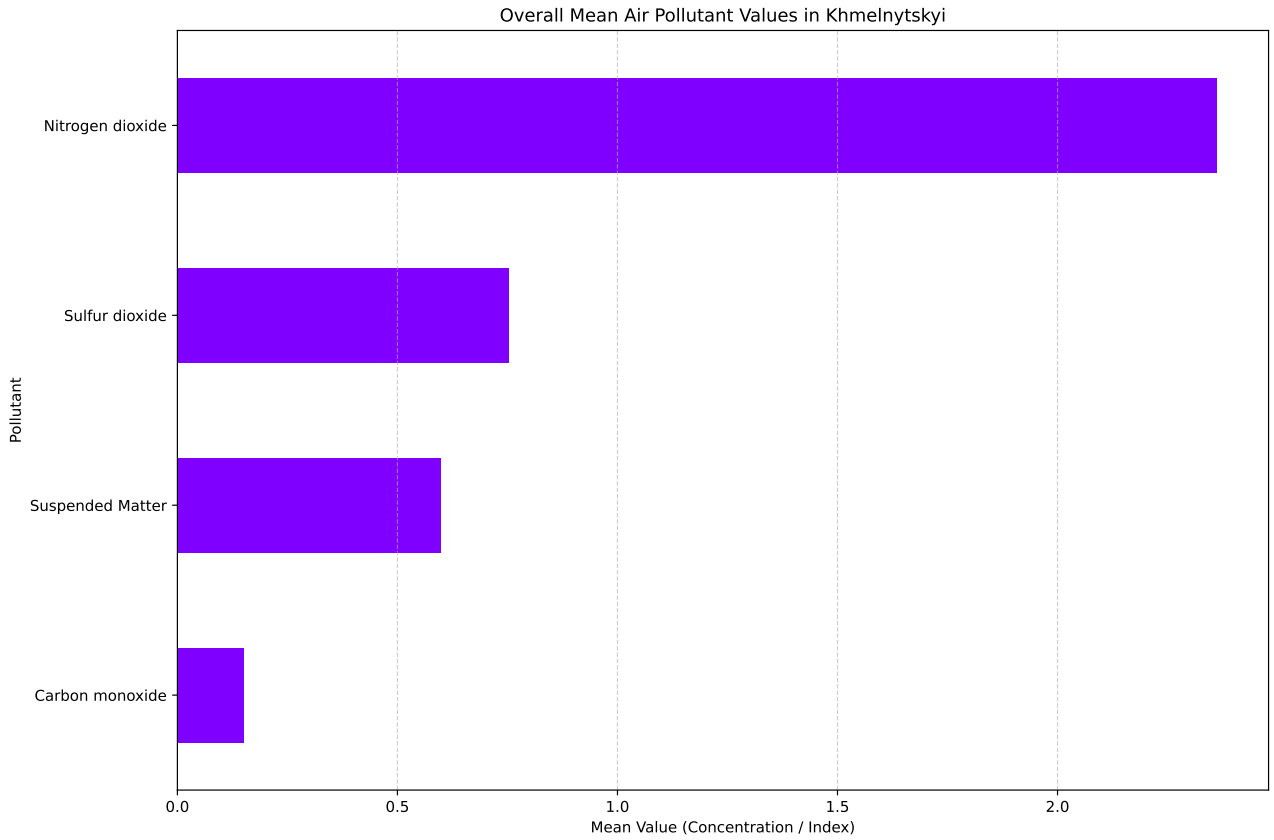


Рис. 9: Горизонтальний стовпчиковий графік середніх значень забруднюючих речовин в Хмельницькому. Як бачимо, найбільш концентрованими забруднюючими речовинами є nitrogen dioxide, formaldehyde та suspanded matter, які мають найбільші середні значення серед усіх забруднюючих речовин. Це може свідчити про наявність промислових викидів або інших джерел забруднення, які потребують додаткового дослідження.

2.3.3 Показники по роках

Також можна відобразити показники по роках для кожної забруднюючої речовини, щоб побачити, як змінюється рівень забруднення протягом часу. Для цього ми зобразимо *box plot* – графік, що показує середнє значення, 1-3 квантиль, а також min/max значення. Він побудований для кожної забруднюючої речовини за допомогою наступного коду на Python:

```
def plot_yearly_distributions(df, pollutant_columns, output_dir="plots/yearly_comparison
    """
    Generates and saves box plots showing the distribution of each pollutant
    level for each year.
    """

    print("\nGenerating yearly distribution box plots...")
```

```
output_path = Path(output_dir)
output_path.mkdir(parents=True, exist_ok=True)
print(f"Ensured directory exists: {output_path}")

df_with_year = df.copy()
df_with_year.index = pd.to_datetime(df_with_year.index)
df_with_year = df_with_year[df_with_year.index.notna()]
df_with_year['Year'] = df_with_year.index.year

for col in pollutant_columns:
    plt.figure(figsize=(12, 7))

    if col not in df_with_year or df_with_year[col].isnull().all():
        print(f"No data for pollutant '{col}'. Skipping yearly box plot.")
        plt.close()
        continue

    # Again, setting some styles and stuff
    sns.boxplot(x='Year', y=col, data=df_with_year, palette="viridis")
    plt.title(f'Yearly Distribution of {col}', fontsize=16)
    plt.xlabel('Year', fontsize=12)
    plt.ylabel(f'{col} Level', fontsize=12)
    plt.grid(axis='y', linestyle='--', alpha=0.6)
    plt.tight_layout()

    safe_col_name = col.replace(' ', '_').replace('/', '_').lower()
    full_plot_path = output_path / f"boxplot_yearly_{safe_col_name}.pdf"

    try:
        plt.savefig(full_plot_path, dpi=300, format='pdf', bbox_inches='tight')
        print(f"Saved yearly box plot to: {full_plot_path}")
    except Exception as e:
        print(f"Error saving box plot for {col}: {e}")

    plt.close()
```

Результати зображено на Рисунку 10.

2.4 Спектральний аналіз

Якщо нам наданий набір даних $X_1, \ldots, X_n$, то дискретне перетворення Фур'є (DFT) визначається як набір наступних комплексних чисел:

$$\widehat{X}_k = \sum_{j \in [n]} X_j e^{-2\pi i j k / n}, \quad k \in [n],$$

де через $[n]$ ми позначили $\{0, \ldots, n-1\}$. Ці числа є коефіцієнтами Фур'є, які містить інформацію про частотний склад компоненти $\frac{k}{n}$. Амплітуда та фаза

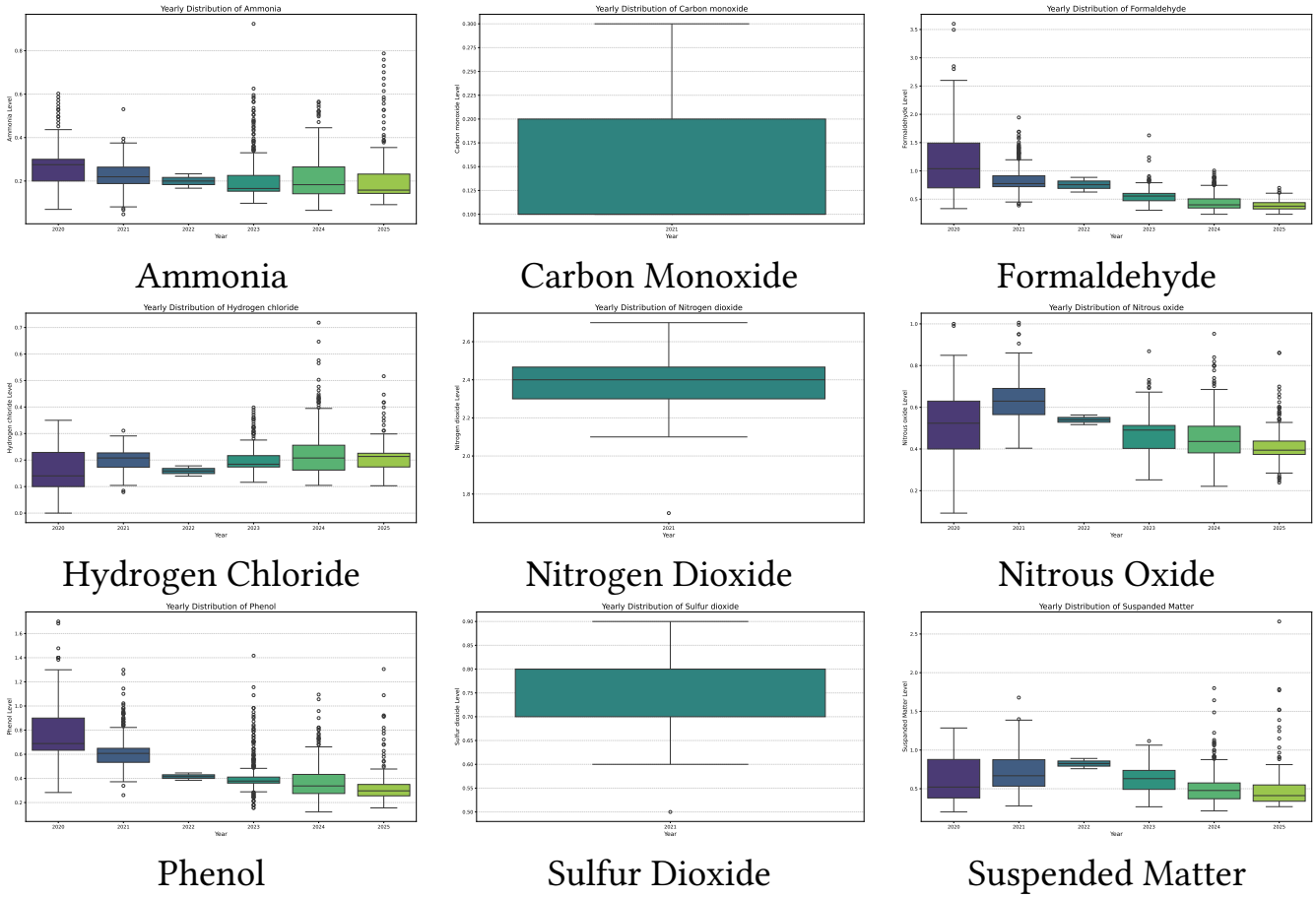


Рис. 10: *Box plot* для кожної забруднюючої речовини по роках.

кожного коефіцієнта Фур'є визначаються як:

$$A_k = \frac{2|\widehat{X}_k|}{n}, \quad \phi_k = \arg(\widehat{X}_k).$$

Амплітудний спектр в такому разі будується як набір пар $\{\left(\frac{k}{n}, A_k\right)\}_{k\in[n]}$, а відповідно фазовий спектр як набір пар $\{\left(\frac{k}{n}, \phi_k\right)\}_{k\in[n]}$. Висока амплітуда буде показувати на що, що в даних присутня сильна гармоніка $\frac{k}{n}$, а фаза показує початковий зсув цієї гармоніки. Напишемо код для цього:

```
DEFAULT_NUM_HARMONICS: int = 9  # Default number of harmonics to analyze


def perform_fft_analysis(df, pollutant_columns, num_harmonics: int = DEFAULT_NUM_HARMONI
    """
    Performs FFT on each pollutant time series to find amplitudes and phases
    of the first N harmonics (excluding the 0th component).
    """

    results = {}

    print(f"Performing FFT for first {num_harmonics} harmonics...")

    for col in pollutant_columns:
        data_series = df[col].dropna()
        N = len(data_series)

        # Check if there's enough data
        if N < (num_harmonics + 1) * 2: # Need at least ~2 points per harmonic
            print(f"Warning: Not enough data for {col} ({N} points). Skipping FFT.")
            continue

        try:
            # Perform FFT
            yf = np.fft.fft(data_series.values)

            # Calculate amplitudes and phases for the first N harmonics (indices 1 to N)
            # Amplitude = 2.0/N * |Y(f)| for f > 0
            amplitudes = 2.0 / N * np.abs(yf[1 : num_harmonics + 1])
            phases = np.angle(yf[1 : num_harmonics + 1]) # Phase in radians

            results[col] = {'Amplitude': amplitudes, 'Phase': phases}

        except Exception as e:
            print(f"Error performing FFT for {col}: {e}")
            continue
```

```
    if not results:
        print("FFT analysis failed for all pollutants.")
        return None, None

    # Convert results to DataFrames for easier plotting
    amp_df = pd.DataFrame({k: v['Amplitude'] for k, v in results.items()})
    phase_df = pd.DataFrame({k: v['Phase'] for k, v in results.items()})

    # Set index to '1' through 'num_harmonics'
    harmonic_index = range(1, num_harmonics + 1)
    amp_df.index = harmonic_index
    phase_df.index = harmonic_index
    amp_df.index.name = "Harmonic"
    phase_df.index.name = "Harmonic"

    print("FFT analysis complete.")
    return amp_df, phase_df
```

Результати перетворення Фур'є для кожної забруднюючої речовини збережено в CSV-файли `fft_amplitudes.csv` та `fft_phases.csv`. Візуалізуємо ці результати за допомогою наступного коду на Python:

```
def _plot_spectrum_bars(spec_df, title, ylabel, filename, output_dir):
    """
    Helper function to plot amplitude or phase spectrum as a grouped bar plot.
    """

    if spec_df is None or spec_df.empty:
        print(f"No spectrum data provided for {title}. Skipping plot.")
        return

    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)

    num_harmonics, num_pollutants = spec_df.shape
    fig_width = max(12, num_harmonics * num_pollutants * 0.2)

    _ = spec_df.plot(kind='bar', figsize=(fig_width, 7), width=0.8)

    plt.title(title, fontsize=16)
    plt.xlabel('Harmonic Index (Frequency Component)', fontsize=12)
    plt.ylabel(ylabel, fontsize=12)
    plt.xticks(rotation=0)
    plt.legend(title='Pollutants', bbox_to_anchor=(1.02, 1), loc='upper left')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.tight_layout(rect=[0, 0, 0.88, 1])

    full_plot_path = output_path / f"{filename}.pdf"
```

```
    try:
        plt.savefig(full_plot_path, dpi=300, format='pdf', bbox_inches='tight')
        plt.close()
        print(f"Saved spectrum plot to: {full_plot_path}")
    except Exception as e:
        print(f"Error saving spectrum plot {full_plot_path}: {e}")


def plot_amplitude_spectrum(amp_df, output_dir="plots"):
    """
    Plots the amplitude spectrum as a grouped bar plot.
    """

    _plot_spectrum_bars(amp_df,
                        'Amplitude Spectrum of Pollutant Oscillations',
                        'Amplitude',
                        'amplitude_spectrum',
                        output_dir)


def plot_phase_spectrum(phase_df, output_dir="plots"):
    """
    Plots the phase spectrum as a grouped bar plot.
    """

    _plot_spectrum_bars(phase_df,
                        'Phase Spectrum of Pollutant Oscillations',
                        'Phase (Radians)',
                        'phase_spectrum',
                        output_dir)
```

Результати спектрального аналізу зображено на Рисунку 11.

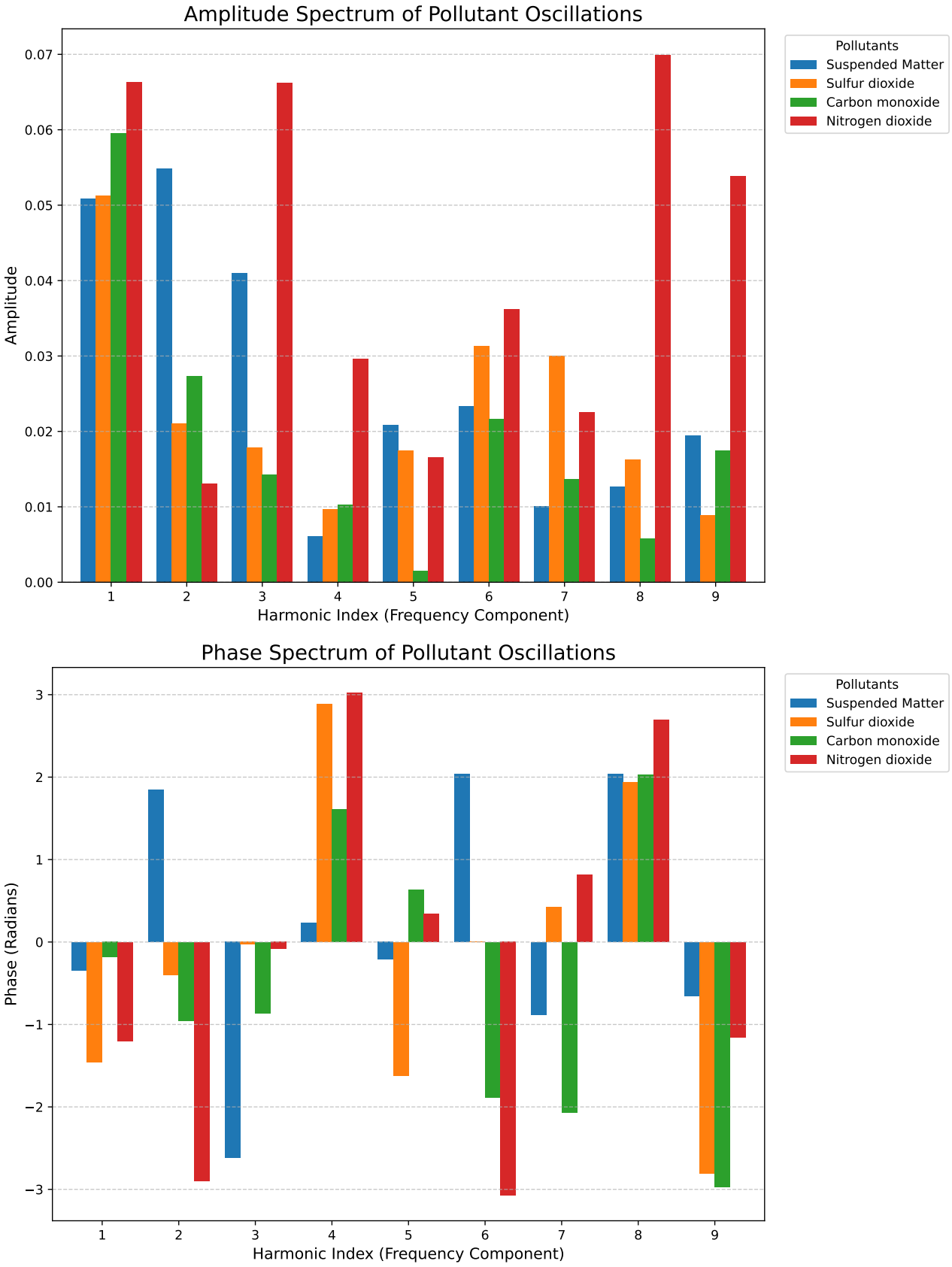


Рис. 11: Групові стовпчикові графіки амплітудного та фазового спектрів забруднюючих речовин в Хмельницькому.

2.5 Фазовий аналіз динамічної системи

Щоб проаналізувати фазовий портрет забруднюючих речовин, ми можемо знайти похідні від послідовних значень концентрації забруднюючих речовин та побудувати графік залежності похідної від значення забруднюючої речовини (а також другої похідної, за необхідності). Скажімо, якщо маємо значення $X_1, \ldots, X_n$, то похідна $\dot{X}_i \approx \frac{X_{i+1} - X_{i-1}}{2\tau}$ де τ — це інтервал між послідовними значеннями. Другу похідну можна обрахувати як $\ddot{X}_i \approx \frac{X_{i+1} - 2X_i + X_{i-1}}{\tau^2}$. Обидві формули називаються **центральними різницями** і є наближеннями похідних для дискретних даних. На щастя, в pandas вже є реалізовані методи для обрахунку похідних, тому ми можемо просто викликати їх на даних. Маємо наступний код на Python:

```python
def plot_phase_analysis(
    df,
    pollutant_columns,
    output_dir="plots/phase_analysis"
) -> None:
    """
    Generates and saves phase analysis plots (derivatives vs. value/derivatives)
    for each pollutant in separate files.
    """

    if df.empty:
        print("DataFrame is empty. No phase plots will be generated.")
        return

    # Ensure the output directory exists
    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)
    print(f"Ensured directory exists: {output_path}")

    # Iterate through each pollutant column
    for col in pollutant_columns:
        # Calculate value and derivatives
        x = df[col]
        dx_dt = df[col].diff()  # 1st derivative (velocity)
        d2x_dt2 = dx_dt.diff() # 2nd derivative (acceleration)

        plot_df = pd.DataFrame({
            'x': x,
            'dx_dt': dx_dt,
            'd2x_dt2': d2x_dt2
        }).dropna()

        # Check if there is data left to plot
        if plot_df.empty:
```

```
        print(f"Not enough data (after differentiation) for {col}. Skipping phase p
        continue

    # Save the derivative data to CSV
    safe_col_name = col.replace(' ', '_').replace('/', '_').lower()
    csv_filename = f"phase_data_{safe_col_name}.csv"
    full_csv_path = output_path / csv_filename
    try:
        plot_df.to_csv(full_csv_path)
        print(f"Saved phase derivative data for {col} to: {full_csv_path}")
    except Exception as e:
        print(f"Error saving phase data CSV for {col}: {e}")

    # Create a figure with 3 subplots for three phase portraits
    fig, axes = plt.subplots(1, 3, figsize=(20, 6)) # 1 row, 3 columns
    fig.suptitle(f'Phase Analysis Plots for {col}', fontsize=16, y=1.03)

    # Plot 1: 1st Der vs. Value (Velocity vs. Position)
    axes[0].scatter(plot_df['x'], plot_df['dx_dt'], s=5, alpha=0.3, color='blue')
    axes[0].set_title('1st Derivative vs. Value')
    axes[0].set_xlabel(f'{col} (Position)')
    axes[0].set_ylabel('1st Derivative (Velocity)')
    axes[0].grid(True, linestyle='--', alpha=0.6)

    # Plot 2: 2nd Der vs. Value (Acceleration vs. Position)
    axes[1].scatter(plot_df['x'], plot_df['d2x_dt2'], s=5, alpha=0.3, color='green')
    axes[1].set_title('2nd Derivative vs. Value')
    axes[1].set_xlabel(f'{col} (Position)')
    axes[1].set_ylabel('2nd Derivative (Acceleration)')
    axes[1].grid(True, linestyle='--', alpha=0.6)

    # Plot 3: 2nd Der vs. 1st Der (Phase Portrait)
    axes[2].scatter(plot_df['dx_dt'], plot_df['d2x_dt2'], s=5, alpha=0.3, color='red
    axes[2].set_title('Phase Portrait (Acceleration vs. Velocity)')
    axes[2].set_xlabel('1st Derivative (Velocity)')
    axes[2].set_ylabel('2nd Derivative (Acceleration)')
    axes[2].grid(True, linestyle='--', alpha=0.6)

    plt.tight_layout(rect=[0, 0.03, 1, 0.98])

    # Prepare filename and save
    safe_col_name = col.replace(' ', '_').replace('/', '_').lower()
    plot_filename = f"phase_analysis_{safe_col_name}.pdf"
    full_plot_path = output_path / plot_filename

    try:
        plt.savefig(full_plot_path, dpi=300, format='pdf', bbox_inches='tight')
        print(f"Saved phase plot to: {full_plot_path}")
```

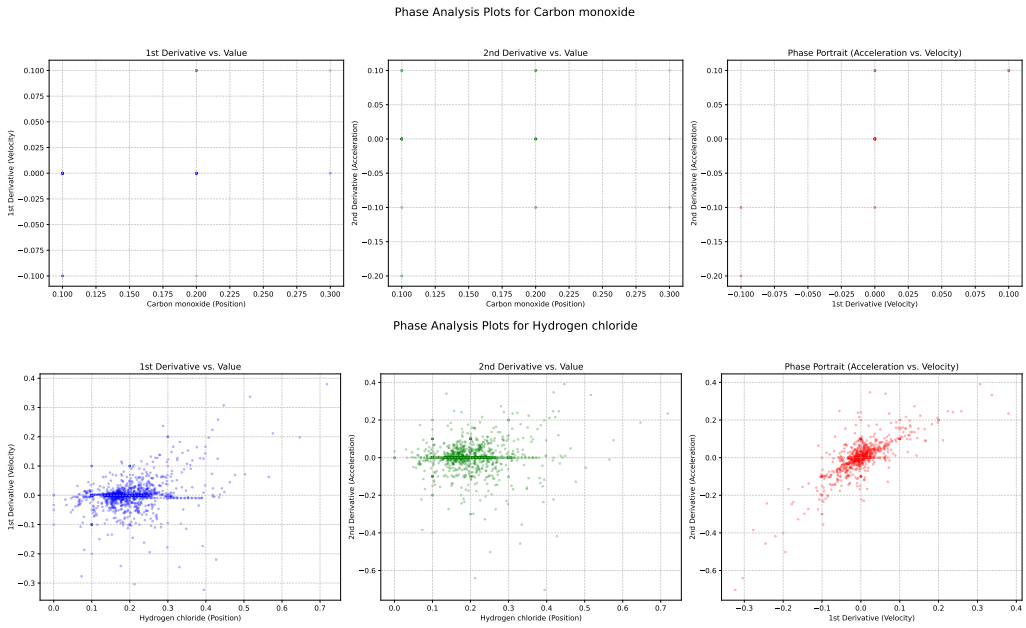


Рис. 12: Фазовий аналіз забруднюючих речовин в Хмельницькому. Графіки зображені лише для carbon monoxide та hydrogen chloride, але аналогічні графіки були побудувані і для інших забруднюючих речовин.

```
        except Exception as e:
            print(f"Error saving phase plot for {col}: {e}")

        plt.close(fig)
```

Часткові результати фазового аналізу зображено на Рисунку 12. Для кожної забруднюючої речовини ми побудували три портрети $(X, \dot{X})$, $(X, \ddot{X})$ та $(\dot{X}, \ddot{X})$, де $X = X(t)$ — це значення забруднюючої речовини як функція часу.

2.6 Кореляційний аналіз

Кореляційний аналіз дозволяє виявити залежності між різними забруднюючими речовинами. Ми можемо обрахувати коефіцієнт кореляції Пірсона для кожної пари забруднюючих речовин, щоб визначити, чи є між ними лінійна залежність. Код для обрахунку коефіцієнтів кореляції виглядає наступним чином:

```
def perform_correlation_analysis(
    df, pollutant_columns, output_csv_dir: str = "csv"
) -> None:
    """
    Calculates the Pearson correlation matrix for the given pollutants
    and saves it to a CSV file.
    """
```

```
    print("Calculating pollutant correlation matrix...")
    if df.empty or not pollutant_columns:
        print("DataFrame is empty or no pollutant columns provided. Skipping correlation
        return None

    try:
        # Select only pollutant columns for correlation
        pollutant_data = df[pollutant_columns]
        correlation_matrix = pollutant_data.corr(method='pearson')
    except Exception as e:
        print(f"Error calculating correlation matrix: {e}")
        return None

    output_path = Path(output_csv_dir)
    output_path.mkdir(parents=True, exist_ok=True)
    correlation_file_path = output_path / "pollutant_correlation_matrix.csv"

    try:
        correlation_matrix.to_csv(correlation_file_path)
        print(f"Pollutant correlation matrix saved to: {correlation_file_path}")
    except Exception as e:
        print(f"Error saving correlation matrix to CSV: {e}")
        # Optionally return matrix even if saving fails, or return None
        # return correlation_matrix
        return None

    return correlation_matrix
```

Ну і відповідно код для візуалізації кореляційної матриці:

```
def plot_correlation_heatmap(correlation_matrix, output_dir="plots") -> None:
    """
    Plots a heatmap of the pollutant correlation matrix.

    Args:
        correlation_matrix (pd.DataFrame): The correlation matrix to plot.
        output_dir (str): Directory to save the heatmap PDF.
    """

    print("Generating correlation heatmap...")
    if correlation_matrix is None or correlation_matrix.empty:
        print("Correlation matrix is empty or None. Skipping heatmap plot.")
        return

    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)
    heatmap_file_path = output_path / "pollutant_correlation_heatmap.pdf"
```

```
plt.figure(figsize=(max(10, len(correlation_matrix.columns) * 0.8), max(8, len(corre
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt=".2f",
    linewidths=.5,
    vmin=-1, vmax=1,
    cbar_kws={'label': 'Correlation Coefficient'}
)
plt.title('Correlation Matrix Heatmap', fontsize=16)
plt.xticks(rotation=45, ha="right", fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()

try:
    plt.savefig(heatmap_file_path, dpi=300, format='pdf')
    print(f"Correlation heatmap saved to: {heatmap_file_path}")
except Exception as e:
    print(f"Error saving correlation heatmap: {e}")
plt.close()
```

Результат кореляційного аналізу вийшов цікавий: див. Рисунок 13.

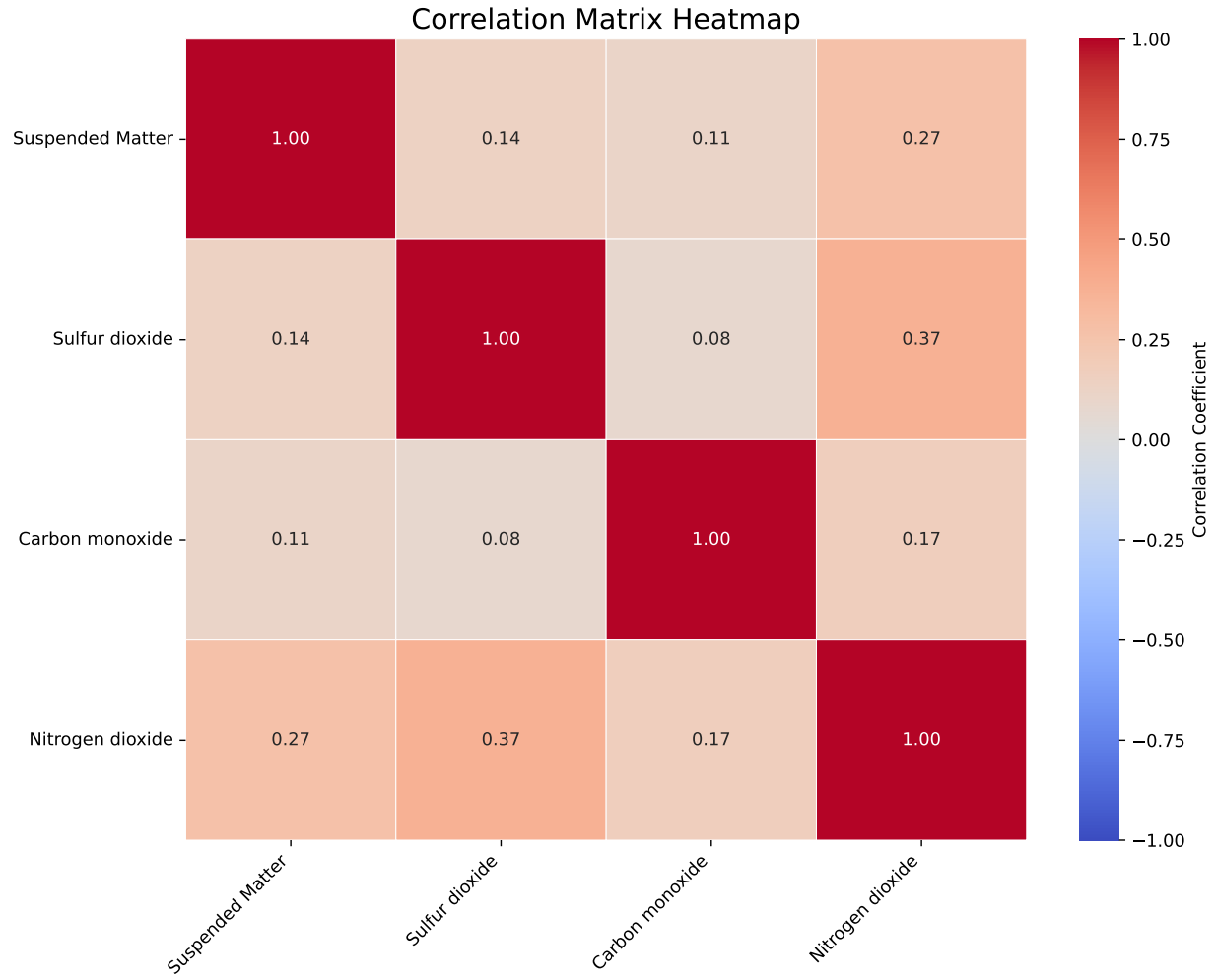


Рис. 13: Кореляційна матриця забруднюючих речовин в Хмельницькому. Як бачимо, найбільша кореляція спостерігається між formaldehyde та sulfur dioxide: а саме, 0.51, а також між phenol та formaldehyde, що дорівнює 0.5. Це може свідчити про те, що ці забруднюючі речовини мають спільні джерела викидів або взаємодіють в атмосфері.

З цікавості, ми вирішили побудувати лінію регресії між формальдегідом та діоксидом сірки, щоб побачити, як змінюється рівень формальдегіду в залежності від рівня діоксиду сірки. Маємо Рисунок 14.

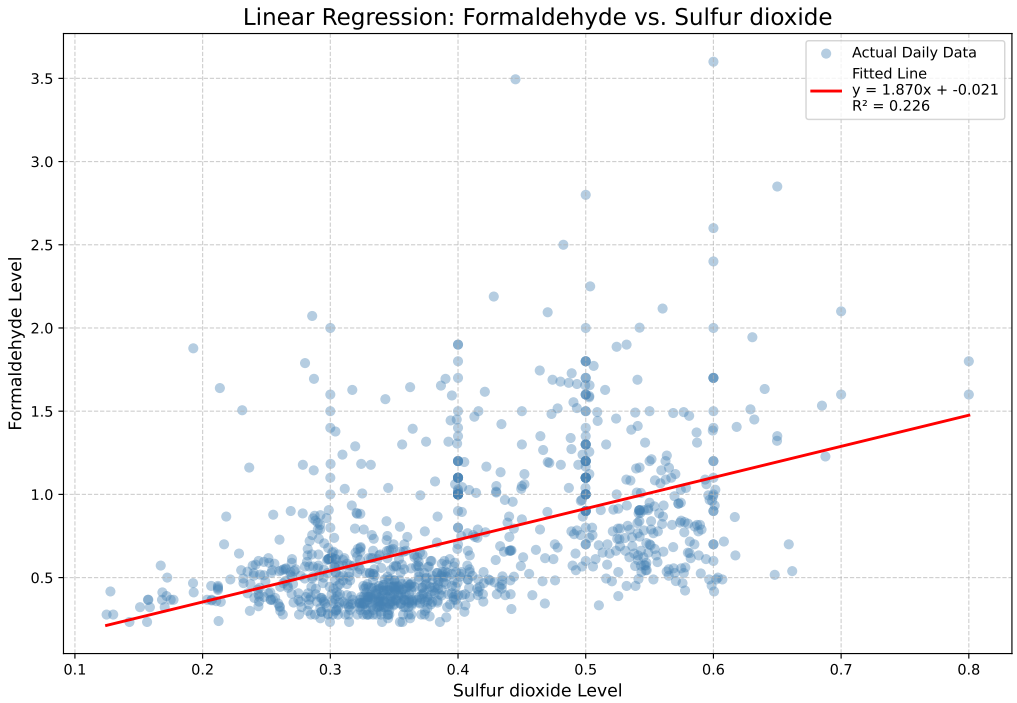


Рис. 14: Лінія регресії $y(x) \approx 1.87x - 0.021$ між формальдегідом та діоксидом сірки в Хмельницькому. Як бачимо, рівень формальдегіду зростає з ростом рівня діоксиду сірки.